LAPORAN TUGAS

WEB SERVER
(TIF 330805)

MINGGU KE 10



ACARA 16

DIBUAT OLEH:

NAMA: CINTYA AZWA SAFRINA R.H

NIM: E41250631

GOL: A

PROGRAM STUDI DI LUAR KAMPUS UTAMA (PSDKU) POLITEKNIK NEGERI JEMBER
TEKNIK INFORMATIKA KAB. NGANJUK JL. JENDRAL GATOT SUBROTO NO 106, KAUMAN, KECAMATAN NGANJUK, KABUPATEN NGANJUK
JURUSAN TEKNOLOGI INFORMASI POLITEKNIK NEGERI JEMBER TAHUN 2026

PENDAHULUAN

Pada Acara 15, API data mahasiswa sudah dapat membaca dan menambah data, tetapi alamat yang dipakai masih berbentuk parameter (`?id=6`) dan operasi ubah serta hapus belum tersedia. Pada praktikum ini API dikembangkan menjadi **RESTful API** yang lengkap.

REST (Representational State Transfer) adalah gaya perancangan API berbasis HTTP. Data dipandang sebagai **resource**, misalnya `mahasiswa`, dan setiap resource memiliki **endpoint** berupa URL, misalnya `/api/mahasiswa` untuk seluruh data dan `/api/mahasiswa/8` untuk mahasiswa dengan id 8. Operasi yang dilakukan ditentukan oleh **HTTP method**, bukan oleh nama URL, sehingga satu URL yang sama dapat menghasilkan aksi berbeda.

Method	Endpoint	Fungsi
GET	/api/mahasiswa	Mengambil seluruh data mahasiswa
GET	/api/mahasiswa/{id}	Mengambil satu mahasiswa berdasarkan id
POST	/api/mahasiswa	Menambahkan mahasiswa baru
PUT	/api/mahasiswa/{id}	Mengubah data mahasiswa
DELETE	/api/mahasiswa/{id}	Menghapus mahasiswa

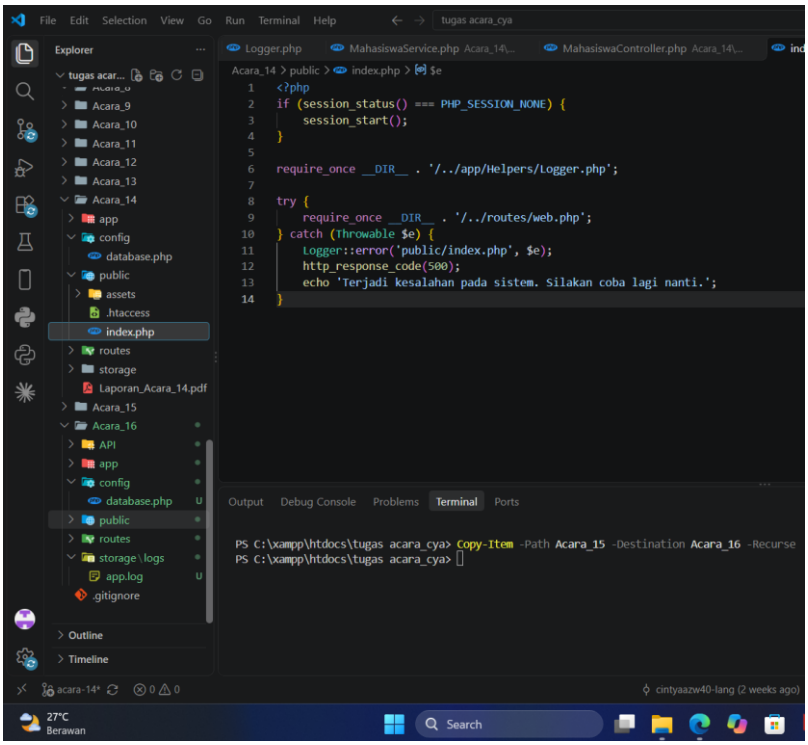
Komunikasi antara client dan API terdiri atas **request** (method, URL, header, dan body) dan **response** (kode status HTTP dan isi dalam format JSON). Kode status yang dipakai: 200 (berhasil), 201 (data berhasil dibuat), 400 (request tidak valid), 404 (data atau endpoint tidak ditemukan), 422 (data tidak lolos validasi), dan 500 (error pada server). Pengujian dilakukan dengan **Postman**, yaitu aplikasi untuk membuat dan mengirim request HTTP ke API tanpa harus membuat tampilan (frontend) terlebih dahulu.

IMPLEMENTASI

1. Menyiapkan branch dan folder Acara_16

```
git checkout -b acara-16
Copy-Item -Path Acara_15 -Destination Acara_16 -Recurse
```

Pekerjaan dilakukan pada branch terpisah `acara-16`. Folder `Acara_15` disalin menjadi `Acara_16` sebagai dasar pengembangan, karena Controller, Service, dan Repository dari praktikum sebelumnya akan dipakai kembali.



Gambar 1. Folder Acara_16 hasil penyalinan dari Acara_15

2. Mengembangkan MahasiswaApiController

`MahasiswaApiController` dilengkapi dengan method `show()`, `update()`, dan `destroy()`. Ringkasan perilaku tiap method:

Method	Alur	Respons
index()	Mengambil data dari Repository (opsional `?q=` untuk pencarian)	200
show($id)	Mencari data berdasarkan id	200 atau 404
store()	Membaca body JSON, mengisi nilai bawaan, memanggil `Service::create()`	201, 400, atau 422
update($id)	Memeriksa id, membaca body JSON, memanggil `Service::update()`	200, 400, 404, atau 422
destroy($id)	Memeriksa id, memanggil `Service::delete()`	200 atau 404

Method	Alur	Respons
Semua method	Error tak terduga dicatat ke `app.log` oleh `Logger`	500 (pesan umum)

```php
<?php
require_once __DIR__ . '/../Repositories/MahasiswaRepository.php';
require_once __DIR__ . '/../Services/MahasiswaService.php';
require_once __DIR__ . '/../Entities/Mahasiswa.php';
require_once __DIR__ . '/../Helpers/Logger.php';

class MahasiswaApiController
{
    private MahasiswaRepository $repo;
    private MahasiswaService $service;

    public function __construct(MahasiswaRepository $repo, MahasiswaService $service)
    {
        $this->repo = $repo;
        $this->service = $service;
    }

    private function json(int $status, bool $success, string $message, $data = null): void
    {
        http_response_code($status);
        header('Content-Type: application/json; charset=utf-8');
        echo json_encode(
            ['success' => $success, 'message' => $message, 'data' => $data],
            JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES
        );
    }

    private function format($row): array
    {
        $prodiNama = null;
        $mahasiswa = $row;

        if (is_array($row) && isset($row['mahasiswa'])) {
            $mahasiswa = $row['mahasiswa'];
            $prodiNama = $row['prodi_nama'] ?? null;
        }

        return [
            'id' => $mahasiswa->getId(),
            'nim' => $mahasiswa->getNim(),
            'nama' => $mahasiswa->getNama(),
            'email' => $mahasiswa->getEmail(),
            'prodi_id' => $mahasiswa->getProdiId(),
            'prodi_nama' => $prodiNama,
            'angkatan' => $mahasiswa->getAngkatan(),
            'status' => $mahasiswa->getStatus(),
        ];
    }

    private function readJson(): ?array
    {
        $input = json_decode(file_get_contents('php://input'), true);

        if (json_last_error() !== JSON_ERROR_NONE || !is_array($input)) {
            return null;
        }

        return $input;
    }
```

```
// GET /api/mahasiswa
public function index(): void
{
    try {
        $keyword = trim($_GET['q'] ?? '');
        $rows = $this->repo->all($keyword !== '' ? $keyword : null);
        $data = array_map([$this, 'format'], $rows);

        $this->json(200, true, 'Data berhasil diambil', $data);
    } catch (Throwable $e) {
        Logger::error('MahasiswaApiController::index', $e);
        $this->json(500, false, 'Terjadi kesalahan pada server');
    }
}

// GET /api/mahasiswa/{id}
public function show(int $id): void
{
    try {
        $mahasiswa = $this->repo->find($id);

        if ($mahasiswa === null) {
            $this->json(404, false, 'Data mahasiswa tidak ditemukan');
            return;
        }

        $this->json(200, true, 'Data berhasil diambil', $this->format($mahasiswa));
    } catch (Throwable $e) {
        Logger::error('MahasiswaApiController::show', $e);
        $this->json(500, false, 'Terjadi kesalahan pada server');
    }
}

// POST /api/mahasiswa
public function store(): void
{
    try {
        $input = $this->readJson();

        if ($input === null) {
            $this->json(400, false, 'Body harus berupa JSON yang valid');
            return;
        }

        $input = array_merge([
            'prodi_id' => 1,
            'angkatan' => (int) date('Y'),
            'status' => 'aktif',
        ], $input);

        $result = $this->service->create($input);

        if ($result['success']) {
            $this->json(201, true, 'Data mahasiswa berhasil ditambahkan');
            return;
        }

        $this->json(422, false, 'Data tidak valid', $result['errors']);
    } catch (Throwable $e) {
        Logger::error('MahasiswaApiController::store', $e);
        $this->json(500, false, 'Terjadi kesalahan pada server');
    }
}
```

```
    // PUT /api/mahasiswa/{id}
    public function update(int $id): void
    {
        try {
            $mahasiswa = $this->repo->find($id);

            if ($mahasiswa === null) {
                $this->json(404, false, 'Data mahasiswa tidak ditemukan');
                return;
            }

            $input = $this->readJson();

            if ($input === null) {
                $this->json(400, false, 'Body harus berupa JSON yang valid');
                return;
            }

            $input = array_merge([
                'prodi_id' => $mahasiswa->getProdiId(),
                'angkatan' => $mahasiswa->getAngkatan(),
                'status' => $mahasiswa->getStatus(),
            ], $input);

            $result = $this->service->update($id, $input);

            if ($result['success']) {
                $this->json(200, true, 'Data mahasiswa berhasil diubah');
                return;
            }

            $this->json(422, false, 'Data tidak valid', $result['errors']);
        } catch (Throwable $e) {
            Logger::error('MahasiswaApiController::update', $e);
            $this->json(500, false, 'Terjadi kesalahan pada server');
        }
    }

    // DELETE /api/mahasiswa/{id}
    public function destroy(int $id): void
    {
        try {
            if ($this->repo->find($id) === null) {
                $this->json(404, false, 'Data mahasiswa tidak ditemukan');
                return;
            }

            $result = $this->service->delete($id);

            if ($result['success']) {
                $this->json(200, true, 'Data mahasiswa berhasil dihapus');
                return;
            }

            $this->json(500, false, 'Data gagal dihapus');
        } catch (Throwable $e) {
            Logger::error('MahasiswaApiController::destroy', $e);
            $this->json(500, false, 'Terjadi kesalahan pada server');
        }
    }
}
```

Pada `store()`, kolom `prodi_id`, `angkatan`, dan `status` diisi nilai bawaan jika tidak dikirim, sehingga body cukup berisi `nim`, `nama`, dan `email`. Pada `update()`, ketiga kolom itu diambil

dari data lama. Kolom `nim` dan `nama` tetap wajib dikirim sehingga body yang tidak lengkap ditolak oleh validasi Service. Seluruh aturan validasi (NIM wajib, angka, dan unik; nama wajib; email valid; prodi tersedia) tetap berada di `MahasiswaService`, sehingga aturan yang sama dipakai oleh halaman web dan API.

3. Menambahkan route RESTful pada routes/web.php

Tiga route baru ditambahkan untuk endpoint dengan `{id}` pada path. Pola `{id}` pada router diubah menjadi `(\d+)`, sehingga angka pada URL diteruskan sebagai parameter method Controller. Method HTTP yang dikirim client (`GET`, `PUT`, `DELETE`) dibandingkan dengan method pada daftar route untuk menentukan method Controller yang dijalankan.

```
['GET',    'api/mahasiswa',          'MahasiswaApiController', 'index',   false],
['POST',   'api/mahasiswa',          'MahasiswaApiController', 'store',   false],
['GET',    'api/mahasiswa/{id}',     'MahasiswaApiController', 'show',    false],
['PUT',    'api/mahasiswa/{id}',     'MahasiswaApiController', 'update',  false],
['DELETE', 'api/mahasiswa/{id}',     'MahasiswaApiController', 'destroy', false],
```

Pada bagian akhir `routes/web.php`, alamat yang diawali `api/` tetapi tidak cocok dengan route mana pun kini dijawab JSON 404, bukan teks biasa.

```
if (strpos($uri, 'api/') === 0) {
    http_response_code(404);
    header('Content-Type: application/json; charset=utf-8');
    echo json_encode([
        'success' => false,
        'message' => 'Endpoint tidak ditemukan',
        'data' => null,
    ]);
    return;
}
```

4. Menguji API menggunakan Postman

Seluruh endpoint diuji melalui Postman dengan alamat dasar `http://localhost/tugas%20acara_cya/Acara_16/public/api/mahasiswa`. Untuk POST dan PUT, body dikirim melalui tab Body dengan pilihan raw dan berformat JSON. Semua request dikumpulkan dalam satu **Postman Collection** (`Acara_16.postman_collection.json`) yang berisi skenario berhasil dan skenario error, sehingga dapat diimpor kembali melalui menu Import.

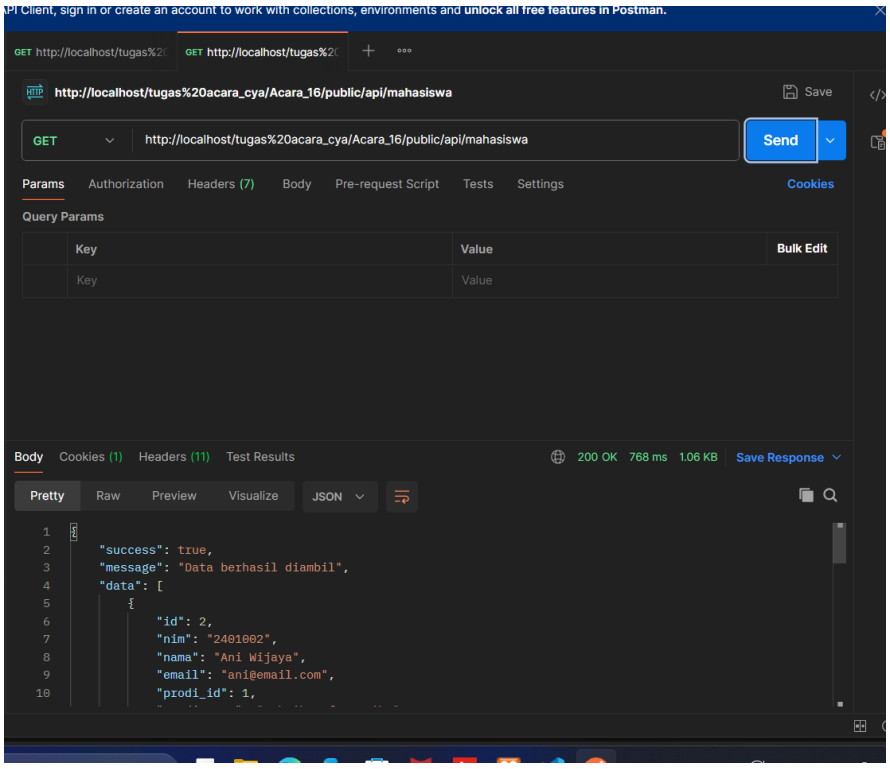
HASIL PENGUJIAN

Ringkasan pengujian seluruh endpoint:

No	Skenario	Endpoint	Status	Hasil
1	GET seluruh mahasiswa	GET /api/mahasiswa	200	Sesuai
2	GET mahasiswa berdasarkan id	GET /api/mahasiswa/2	200	Sesuai
3	POST data valid (Rina)	POST /api/mahasiswa	201	Sesuai
4	PUT ubah data (Rina Aprilia)	PUT /api/mahasiswa/8	200	Sesuai
5	DELETE hapus data	DELETE /api/mahasiswa/8	200	Sesuai
6	GET setelah dihapus	GET /api/mahasiswa/8	404	Sesuai
7	PUT dengan id tidak ditemukan	PUT /api/mahasiswa/9	404	Sesuai
8	PUT tanpa body JSON	PUT /api/mahasiswa/8	400	Sesuai
9	POST dengan NIM kosong	POST /api/mahasiswa	422	Sesuai
10	POST dengan email tidak valid	POST /api/mahasiswa	422	Sesuai
11	POST dengan JSON tidak lengkap	POST /api/mahasiswa	422	Sesuai

a. GET seluruh data (200 OK)

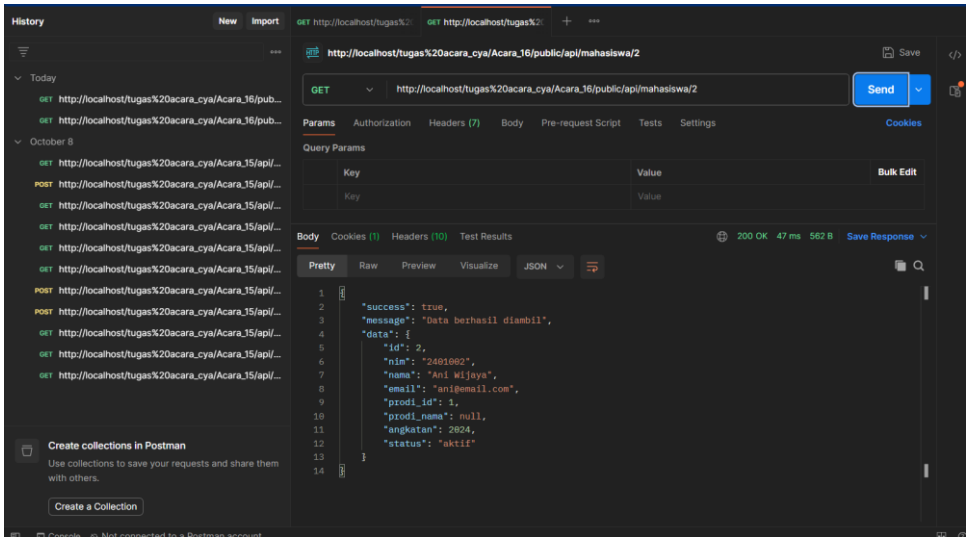
Request GET ke `/api/mahasiswa` mengembalikan seluruh data mahasiswa dalam array JSON.



Gambar 2. GET seluruh mahasiswa, respons 200 OK

b. GET berdasarkan id (200 OK)

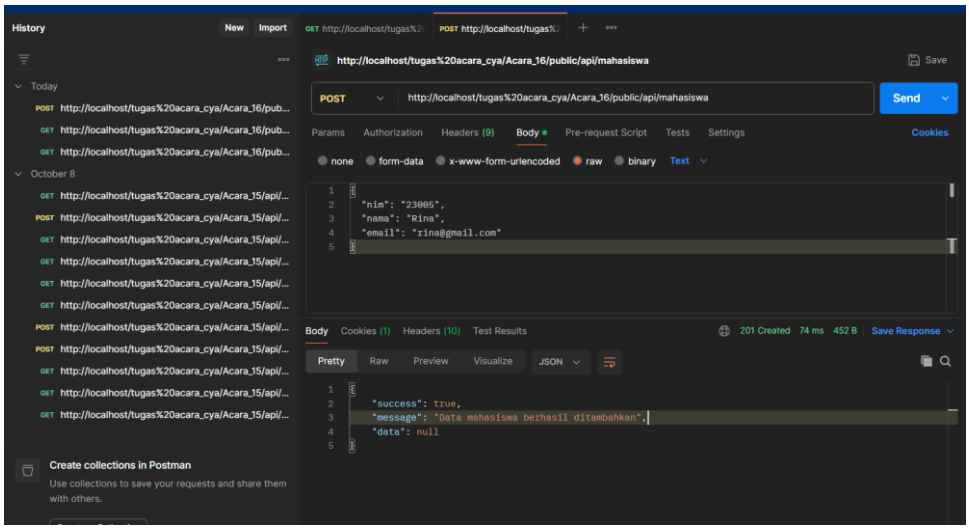
Id ditulis pada path URL (`/api/mahasiswa/2`). Respons berisi satu object data mahasiswa, bukan array.



Gambar 3. GET /api/mahasiswa/2, respons 200 OK

c. POST menambah data (201 Created)

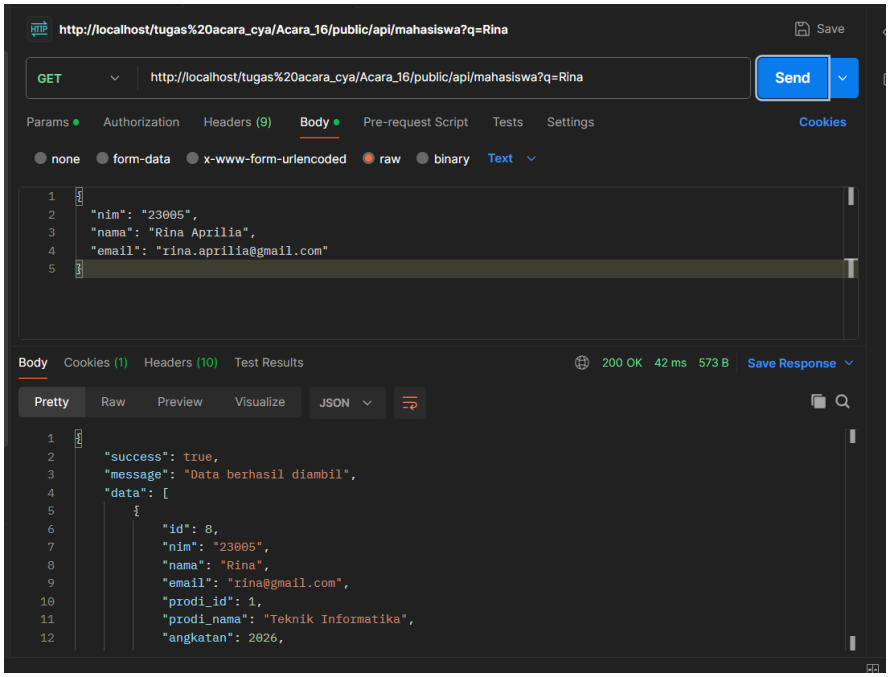
Body JSON berisi `nim` "23005", `nama` "Rina", dan `email` "rina@gmail.com". Server membalas 201 Created.



Gambar 4. POST data valid, respons 201 Created

d. Mencari id data yang baru ditambahkan

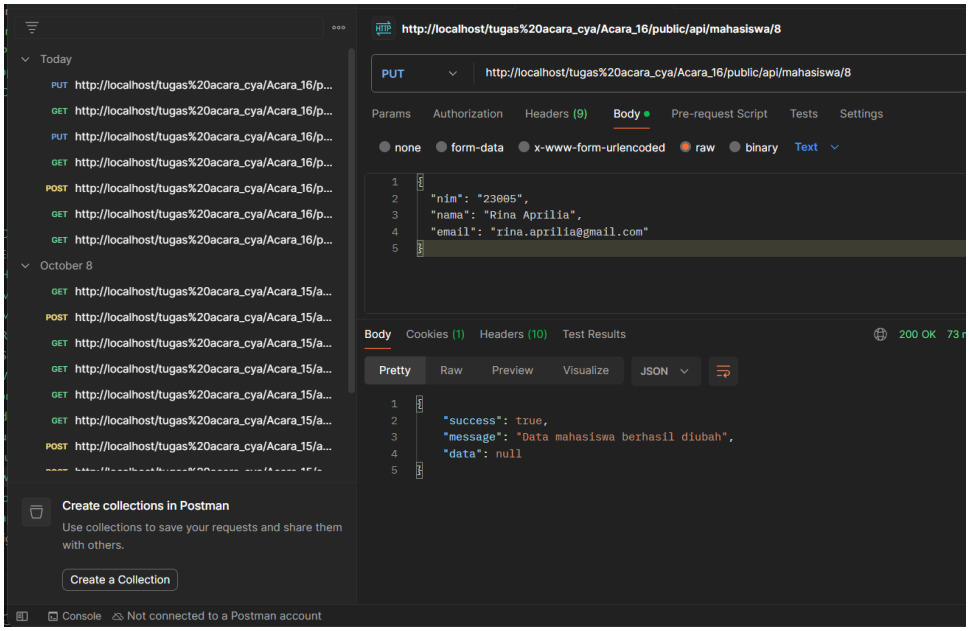
Id Rina dicari dengan parameter pencarian `?q=Rina` dan hasilnya adalah id 8. Id ini dipakai pada pengujian PUT dan DELETE.



Gambar 5. GET /api/mahasiswa?q=Rina, id Rina adalah 8

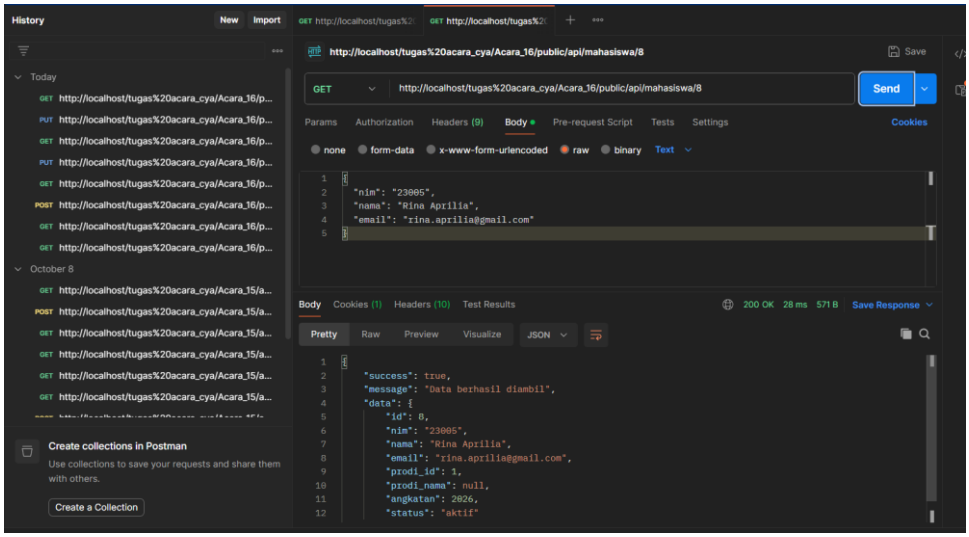
e. PUT mengubah data (200 OK)

Data id 8 diubah menjadi `nama` "Rina Aprilia" dan `email` "rina.aprilia@gmail.com". Server membalas 200 OK.



Gambar 6. PUT /api/mahasiswa/8, respons 200 OK

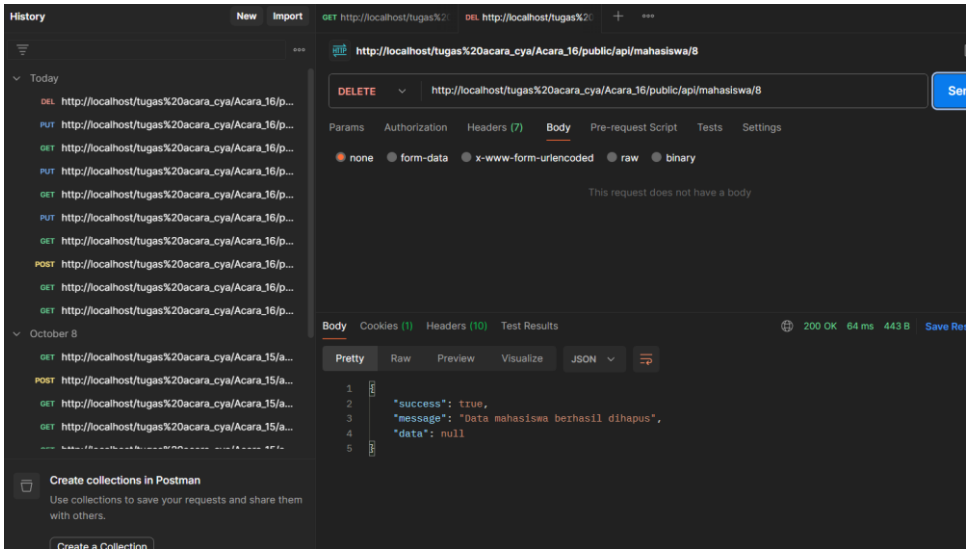
Pengecekan dengan GET menunjukkan data sudah berubah.



Gambar 7. GET /api/mahasiswa/8 setelah PUT, nama menjadi "Rina Aprilia"

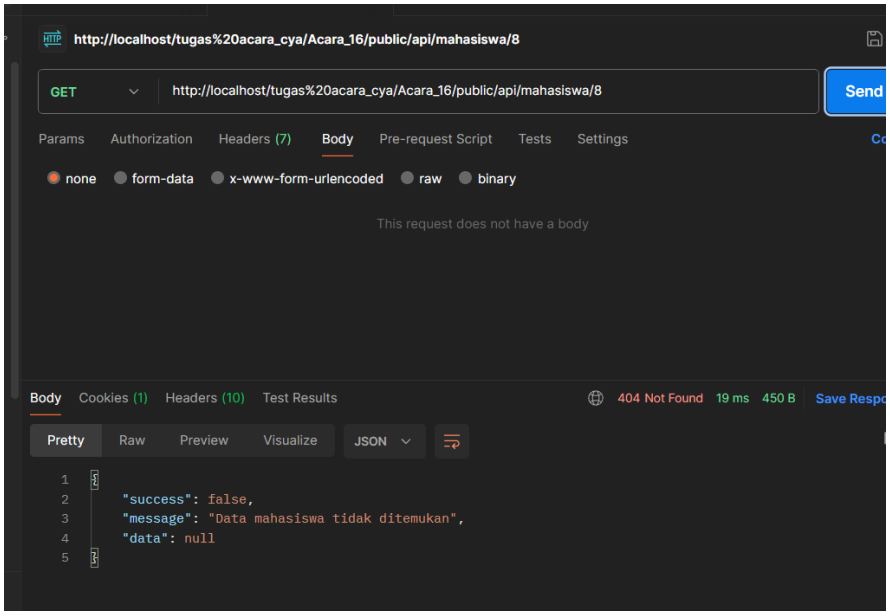
f. DELETE menghapus data (200 OK)

Request DELETE ke `/api/mahasiswa/8` tanpa body menghapus data uji dan dibalas 200 OK.



Gambar 8. DELETE /api/mahasiswa/8, respons 200 OK

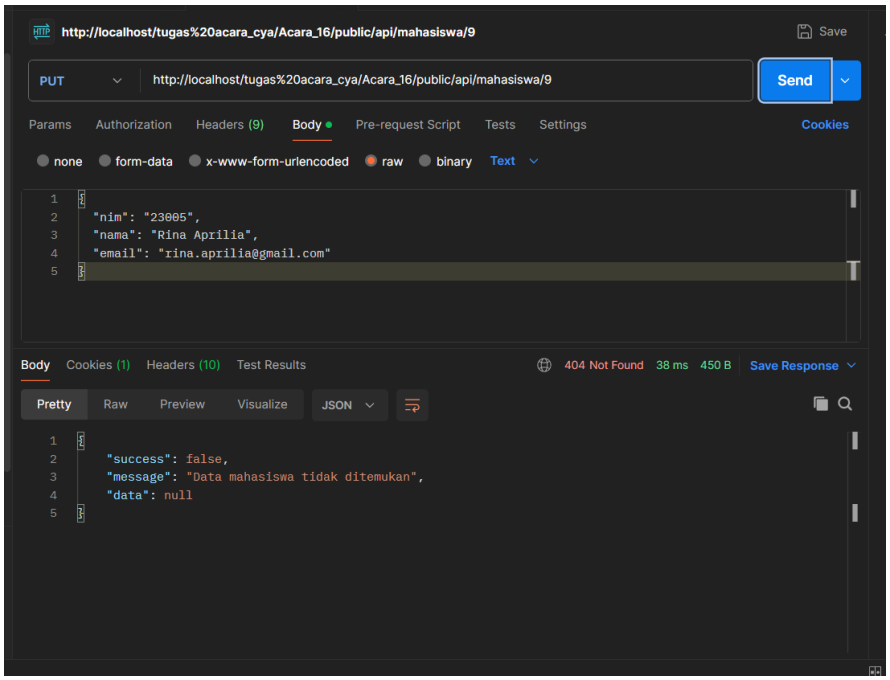
GET ke id yang sama setelah penghapusan menghasilkan 404 Not Found, yang membuktikan data benar-benar hilang.



Gambar 9. GET /api/mahasiswa/8 setelah DELETE, respons 404 Not Found

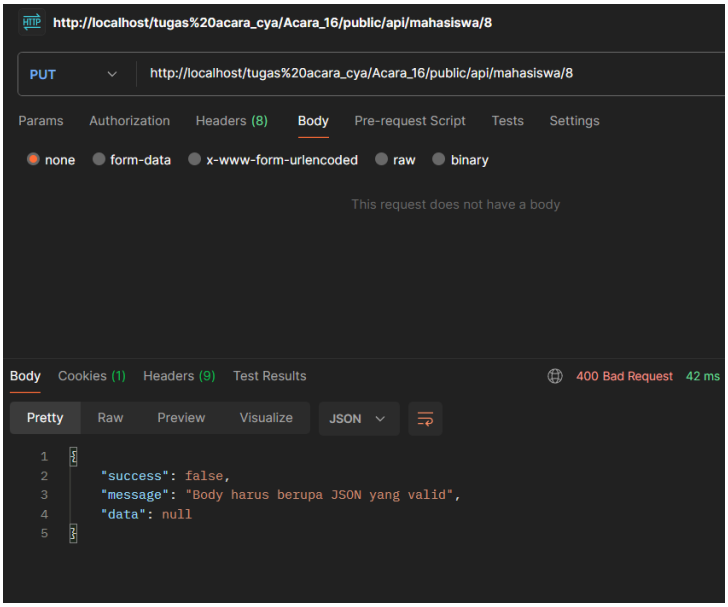
g. Pengujian kondisi error

ID tidak ditemukan. PUT ke id yang tidak ada (id 9) dibalas 404, karena `update()` memeriksa keberadaan data sebelum memproses body.



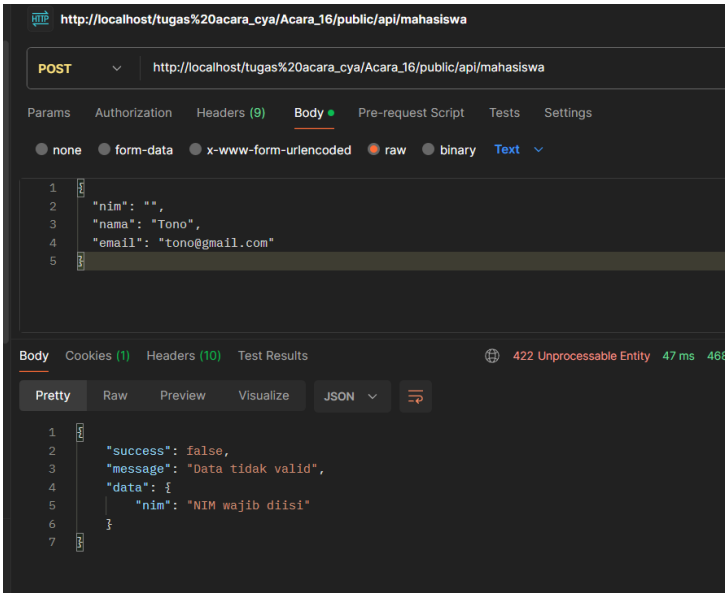
Gambar 10. PUT dengan id tidak ditemukan, respons 404 Not Found

Body bukan JSON. PUT tanpa body ditolak dengan 400 Bad Request.



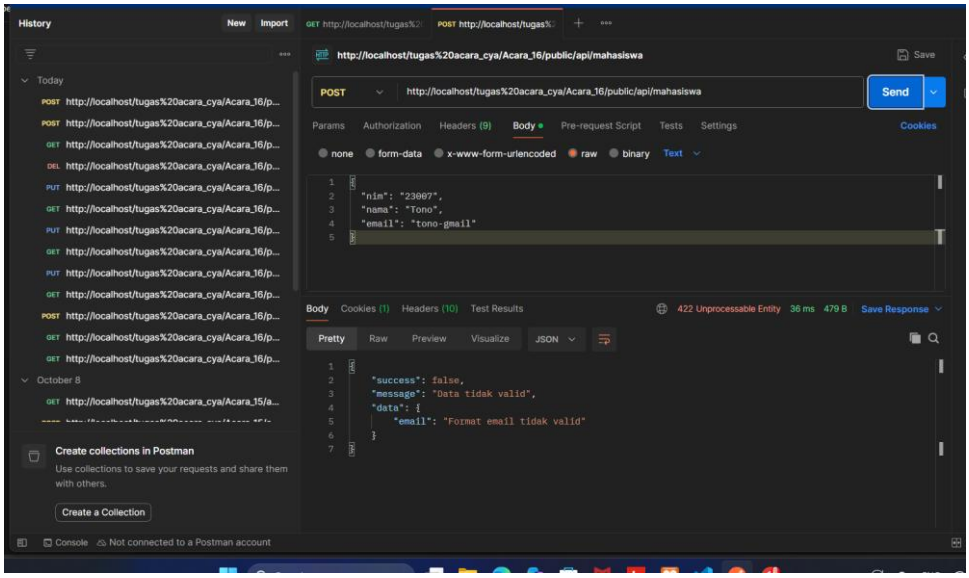
Gambar 11. PUT tanpa body JSON, respons 400 Bad Request

NIM kosong. POST dengan `nim` kosong ditolak dengan 422 dan pesan "NIM wajib diisi".



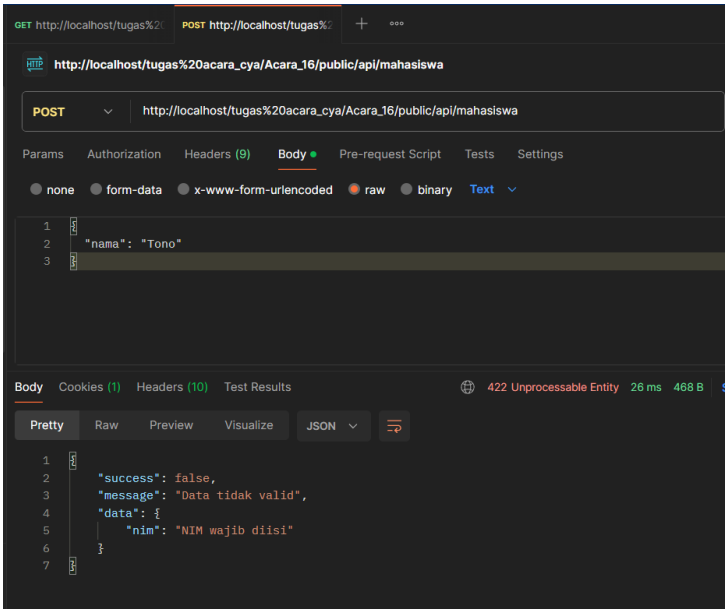
Gambar 12. POST dengan NIM kosong, respons 422

Email tidak valid. POST dengan email "tono-gmail" ditolak dengan 422 dan pesan "Format email tidak valid".



Gambar 13. POST dengan email tidak valid, respons 422

JSON tidak lengkap. POST yang hanya berisi `nama` ditolak dengan 422 karena NIM wajib diisi.



Gambar 14. POST dengan JSON tidak lengkap, respons 422

ANALISIS

Perbedaan terbesar dari Acara 15 adalah cara menentukan operasi. Pada RESTful API, alamat `/api/mahasiswa/8` dapat dipakai untuk tiga tindakan berbeda tergantung HTTP method: GET membaca, PUT mengubah, dan DELETE menghapus. Id tidak lagi dikirim sebagai parameter (`?id=8`) tetapi menjadi bagian dari path, sehingga URL menunjuk langsung ke satu resource. Hal ini dicapai dengan pola `{id}` pada route yang diubah menjadi ekspresi reguler `(\d+)` dan diteruskan sebagai parameter ke method Controller.

Kode status HTTP menyampaikan hasil permintaan dengan tepat. Penambahan data dibalas 201 (Created), bukan 200, karena ada resource baru yang dibuat. Data yang tidak ditemukan dibalas 404, baik pada GET, PUT, maupun DELETE. Body yang bukan JSON dibalas 400, sedangkan JSON yang valid secara format tetapi isinya melanggar aturan (NIM kosong, email salah) dibalas 422 beserta daftar error per kolom. Perbedaan 400 dan 422 membantu client mengetahui apakah masalah ada pada format request atau pada isi datanya.

Pengujian PUT dengan id 9 yang menghasilkan 404 menunjukkan bahwa `update()` memeriksa keberadaan data lebih dulu sebelum membaca body. Pengujian DELETE ditutup dengan GET ke id yang sama yang menghasilkan 404, sehingga terbukti penghapusan benar-benar terjadi di database, bukan hanya mengembalikan pesan sukses.

Pada POST dan PUT, kolom `prodi_id`, `angkatan`, dan `status` tidak wajib dikirim. POST memakai nilai bawaan, sedangkan PUT mempertahankan nilai lama. Kolom `nim` dan `nama` tetap wajib sehingga body yang tidak lengkap menghasilkan 422, sesuai dengan skenario "JSON tidak lengkap" pada modul.

Arsitektur berlapis dari praktikum sebelumnya tetap dipakai: Controller menerima request, Service memvalidasi, dan Repository mengakses database dengan prepared statement. Aturan validasi hanya ditulis satu kali di Service dan dipakai bersama oleh halaman web dan API. Error tak terduga dicatat oleh `Logger` ke `app.log`, sementara client hanya menerima pesan umum dengan kode 500 tanpa detail teknis database. Terakhir, Postman Collection mengumpulkan seluruh skenario sehingga pengujian dapat diulang kapan saja.

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. RESTful API memandang data sebagai resource dengan endpoint berupa URL, dan operasi ditentukan oleh HTTP method (GET, POST, PUT, DELETE).
2. Id resource ditempatkan pada path URL (`/api/mahasiswa/{id}`), sehingga satu endpoint dapat dipakai untuk membaca, mengubah, dan menghapus data.
3. Kode status HTTP yang tepat (200, 201, 400, 404, 422, dan 500) membuat client memahami hasil request tanpa harus membaca isi pesannya.
4. Request dan response menggunakan format JSON dengan struktur seragam (`success`, `message`, `data`).
5. Validasi, prepared statement, dan logging tetap diterapkan sehingga API aman dan tidak menampilkan detail error teknis.
6. Postman dapat dipakai untuk menguji seluruh endpoint tanpa membuat frontend, dan Postman Collection memudahkan pengujian diulang.

DAFTAR PUSTAKA

1. Politeknik Negeri Jember. 2026. Modul Praktikum Workshop SI Web Server (TIF330805), Acara 16: RESTful API + endpoint request/response.

2. Fielding, R. T. 2000. Architectural Styles and the Design of Network-based Software Architectures. Disertasi Doktor, University of California, Irvine.

3. MDN Web Docs. HTTP request methods. https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods

4. MDN Web Docs. HTTP response status codes. https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

5. Postman. Postman Learning Center. https://learning.postman.com